BazzBasic Beginners's Guide

```
 ######   #####  ####### ####### ######   #####  ####### ##  ######
##   ## ##   ##    ###     ###  ##   ## ##   ## ##      ## ##
######  #######   ###     ###   ######  ####### ####### ## ##
##   ## ##   ##  ###     ###    ##   ## ##   ##      ## ## ##
######  ##   ## ####### ####### ######  ##   ## ####### ##  ######
```

Metadata:

Description:
This beginner's guide is written by Kristian Virtanen aka "EkBass" with translation help from Claude.ai for BazzBasic.

Date:
02.04.2026 (dd.mm.yyyy)

License:
This guide is licensed under Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0).

You are free to copy and share this guide in any format, provided you credit the original author. Translations into other languages are permitted. Modifications to the content are not permitted.

Table of Contents

- 1.0 Introducing BASIC
  - 1.1 About BazzBasic
  - 1.2 Who is BazzBasic for?
- 2.1 How To Use BazzBasic
  - 2.2 Install
  - 2.3 First Use
  - 2.4 Using the Built-in IDE
  - 2.5 First Program
- 3.1 Interaction
  - 3.2 INPUT & PRINT
  - 3.3 name$
- 4.1 Data Types
  - 4.2 Variables
  - 4.3 Constants
  - 4.4 Arrays
  - 4.5 Working with Values
  - 4.6 Declaring Without a Value
  - 4.7 Reading Multiple Values From an Array
  - 4.8 Choosing the Right Tool
- 5.0 Program Structure
  - 5.1 A Common Structure
  - 5.2 It's a Convention, Not a Rule
- 6.0 Your First Game
  - 6.1 Doing Things More Than Once with FOR
  - 6.2 Making Decisions with IF
  - 6.3 The Game
  - 6.4 Walking Through It
  - 6.5 Try It Yourself
- 7.0 Reality Check
  - 7.1 A Word From Author
- 8.0 More Loops
  - 8.1 WHILE
  - 8.2 FOR
    - 8.2.1 Exception with FOR variable declaration
  - 8.3 When to Use Which
- 9.0 IF Conditions
  - 9.1 IF - THEN - END IF
    - 9.1.1 Comparison Operators
  - 9.2 IF - THEN - ELSEIF - ELSE - END IF
  - 9.3 Nested IF
  - 9.4 One-liner form
  - 9.5 TRUE and FALSE
- 10.0 Controlling The Screen
  - 10.1 PRINT
    - 10.1.1 Escape sequences
  - 10.2 INPUT and LINE INPUT
  - 10.3 CLS
  - 10.4 LOCATE
  - 10.5 CURPOS
  - 10.6 COLOR
  - 10.7 A Practical Example
- 11.0 Controlling The Program
  - 11.1 GOSUB
    - 11.1.1 Why GOSUB Matters
    - 11.1.2 GOSUB And Program Structure

    - 11.2 GOTO
- 12.0 Putting It All Together
    - 12.1 A Quick Look at WAITKEY
    - 12.2 Russian Roulette
    - 12.3 Walking Through It
- 13.0 User-Defined Functions
    - 13.1 Defining and Calling
    - 13.2 Parameters
    - 13.3 Scope
    - 13.4 The Return Value Rule
    - 13.5 Recursion
    - 13.6 GOSUB vs DEF FN
- 14.0 User Input
    - 14.1 WAITKEY
    - 14.2 INKEY
    - 14.3 KEYDOWN
    - 14.4 Key Constants
    - 14.5 Mouse
    - 14.6 Choosing the Right Tool
- 15.0 Files
    - 15.1 Reading Files
    - 15.2 Writing Files
    - 15.3 Checking and Deleting
    - 15.4 Key-Value Files
    - 15.5 The .env Pattern
    - 15.6 Paths
    - 15.7 A Practical Example
- 16.0 JSON and Arrays
    - 16.1 What Is JSON
    - 16.2 Array to JSON
    - 16.3 JSON to Array
    - 16.4 Nested Data
    - 16.5 JSON Files
    - 16.6 Putting It Together
- 17.0 Network
    - 17.1 HTTPGET
    - 17.2 HTTPPOST
    - 17.3 Headers
    - 17.4 Working With Responses
    - 17.5 Putting It All Together
- 18.0 Graphics and Sound
    - 18.1 Opening a Screen
    - 18.2 Drawing
    - 18.3 Shapes and Images
    - 18.4 SCREENLOCK
    - 18.5 The Game Loop
    - 18.6 Sound
    - 18.7 A Complete Example: Bouncing Ball
    - 18.8 FastTrig For Performance
    - 18.9 What BazzBasic Can Do
        - 18.9.1 2D Field of View in console
        - 18.9.2 Wolfenstein-style 3D Raycaster
        - 18.9.3 Comanche-style Voxel Terrain
- 19.0 Shelling Around
    - 19.1 What You Can Do With It
    - 19.2 Why This Matters
        - 19.2.1 Note About Linux Compatibility
- 20.0 BazzBasic Libraries
    - 20.0.1 What is a Library?
    - 20.0.2 Creating a Library
    - 20.0.3 Naming Convention
    - 20.0.4 Accessing Main Program Constants
    - 20.0.5 Library Rules
    - 20.0.6 Error Handling
    - 20.0.7 Benefits of Libraries
    - 20.0.8 Distributing Libraries
- 21.0 Compiling to EXE
    - 21.0.1 Usage
    - 21.0.2 Required Files for Distribution
    - 21.0.3 Recommended Distribution with Assets
    - 21.0.4 Where to Find SDL2.dll
    - 21.0.5 Notes
    - 21.0.6 .NET Runtime Requirement
- 22.0 AI-Assisted Learning
- 23.0 More Functions and Features
    - 23.1 Mathematical Functions
    - 23.2 String Functions
    - 23.3 Other Functions
    - 23.4 Links
- 24.0 Final Words

1.0 Introducing BASIC

The first version of the BASIC language was Dartmouth BASIC.

It was created by two professors, John G. Kemeny and Thomas E. Kurtz, and the DTSS (Dartmouth Time-Sharing System) behind it provided an interactive programming environment for the university community.

Although other, so-called richer languages (Pascal, Fortran) were already available at the time, Dartmouth BASIC gained considerable attention precisely because of its simplicity.

The golden age of BASIC can be said to be the 80s, and especially the microcomputers of that time such as the **Commodore 64**, **ZX Spectrum**, **Spectravideo 328** and the **MSX** home computer standard.

Although the BASIC languages of the 80s are especially remembered for their almost nightmarish line number requirements, GOTO jump hells, and logically terrifying programs published in hobbyist magazines, they also aroused the curiosity of many who later became professionals in the field.

In the late 80s and well into the late 90s, GWBasic and its successor QBasic successfully continued the BASIC tradition.

The death of BASIC has been predicted for almost as long as it has existed. At the very least, QBasic was supposed to be the "last".

However, FreeBasic, QB64, PureBasic, TrueBasic, YaBasic, etc. have all continued the BASIC story, and many implementations can still be found today.

BASIC is rarely the programming language with which to start building professional software. But it is still an effective way to get started and become a **capable amateur** in the field.

↑ Back to top

1.1 About BazzBasic

BazzBasic represents BASIC languages partly using traditional methods, but it also takes a significant step away from them.

The most significant difference is probably the lack of line numbers altogether and the fact that BazzBasic views and handles program variables differently.

However, the line numbers that divide the BASIC community strongly represent a time that has already passed, by my personal opinion. You may or may not agree, you have right for your opinion too.

Since GOTO/GOSUB are deep in the roots of both BASIC and its users, BazzBasic supports the use of so-called *labels* that can indicate the place in the program where you want it to go.

More on this in the section of the guide dealing with GOTO and GOSUB.

Variables have also always strongly divided the BASIC community. Some believe that variables should not be created or declared in any specific way, because it makes starting programming too complicated. Some people think this leads to a bad start in programming, because it makes mistakes, especially those made by beginners, too easy.

Traditional BASIC does not require variable initialization - almost anything will do.

```
REM Traditional BASIC
a = 1 : b = 2
```

BazzBasic does not tolerate this. It requires each variable to be declared separately.

```
' BazzBasic style
LET a$ = 1, b$ = 2
```

In traditional BASIC languages, the type of the variable's contents has been strongly tied to the suffix at the end of the variable.

```
REM Traditional BASIC
a$ = "String" ' text variable
a% = 123 ' integer variable
```

BazzBasic breaks also this. It does not really care what is stored in the variable, only that it is declared before use.

```
LET a$ = "String"
a$ = 123
```

↑ Back to top

1.2 Who is BazzBasic for?

BazzBasic is especially suitable for those who are just starting to learn programming. It is also an excellent alternative for those who have programmed in an older, a now perhaps outdated BASIC language.

I have deliberately tried to create BazzBasic in a way that quickly and easily provides small experiences of success, which then fuel the curiosity to continue around the next corner.

Like many traditional BASIC languages, BazzBasic can be as simple as one line of code.

```
PRINT "Hello World!"
```

You can share BazzBasic code directly, create tokenized libraries, or even distribute your program as an executable Windows file (.exe) for your friends to run.

BazzBasic is free, its source code is available to everyone, and you have no restrictions on how you distribute the programs you make with it. You can give them away for free, or you can charge for them.

BazzBasic does not aim to be the only language you will ever need. It wants to help you get started, learn programming, maybe find friends in programming communities, and eventually perhaps move on to a more powerful language - just like the BASIC interpreters on home computers in the 80s once did.

↑ Back to top

2.1 How To Use BazzBasic

To download the latest version of BazzBasic, go to this URL with your web browser: https://ekbass.github.io/BazzBasic/

From there, choose *"Downloads"* and download the latest version as a *zip archive* to your computer.

↑ Back to top

2.2 Install

BazzBasic does not require you to install anything. Everything is included in the *zip archive* you just downloaded.

Unpack the archive to a folder of your choice and you are ready to go.

↑ Back to top

2.3 First Use

In the folder, you see *"bazzbasic.exe"*. Just double-click it and the BazzBasic IDE opens on your screen.

↑ Back to top

2.4 Using the Built-in IDE

While the built-in IDE offers everything you need to start programming, there are also BazzBasic syntax files for *Geany, Notepad++, and VS Code*. See the subfolder *extras*.

The IDE itself includes only a minimal set of features. They are common editor options and should not cause any problems.

↑ Back to top

2.5 First Program

Type your first line of BazzBasic code:

```
PRINT "Hello from BazzBasic IDE"
LET a$ = WAITKEY()
```

Save the file as *"test.bas"* and press **F5** to run it.

You should see the text *"Hello from BazzBasic IDE"* on your screen. Press any key to stop the program.

Congratulations.

You have just:

1. Made sure everything works
2. Created your first BazzBasic program

↑ Back to top

3.1 Interaction

Surely you already know how to edit the output of the program you just wrote? Just edit the text inside the quotes and it will work.

If you're feeling playful, try putting *"\t"* in place of a space somewhere just to experiment

e.g.:

```
PRINT "Hello from\tBazzBasic IDE"
LET a$ = WAITKEY()
```

But printing text on screen gets boring pretty quickly, so let's move on and examine the keyword *INPUT*.

↑ Back to top

3.2 INPUT & PRINT

INPUT is a useful way to read input from the user. It also stores the input value in a variable for later use.

Now, write the next program and save it before pressing *F5* again.

```
INPUT "What is your first name?", name$
PRINT "Hello "; name$
LET a$ = WAITKEY()
```

The program now asks your first name and then outputs a greeting with it.

```
What is your first name? <your response here>
Hello <your response here>
```

↑ Back to top

3.3 name$

You noticed an interesting piece of text in the code, *"name$"*? That is a variable. Something that can store text, numbers, and much more.

But there is a whole chapter about them later, so let's leave it at that.

Congratulations.

You have just:

1. Received INPUT from the user
2. Stored your first data to a variable

↑ Back to top

4.1 Data Types

Programs need to remember things. The player's score. A name. Whether a door is open or closed. How many coins are left.

Everything your program remembers lives somewhere in memory, and BazzBasic gives you three ways to work with it: **variables**, **constants**, and **arrays**. Each one has its place, and knowing when to use which one is one of those small skills that quietly makes your code much nicer.

↑ Back to top

4.2 Variables

You already met variables back in chapter 3. Let's get properly introduced.

A variable is a named box. You can put something in it, take it out, swap it for something else. It varies - hence the name.

In BazzBasic, every variable name ends with a `$` sign. This is just how BazzBasic rolls. It's not about the *type* of data inside - BazzBasic doesn't really care about that. The `$` simply means *"this is a variable"*.

```
LET score$ = 0
LET playerName$ = "Alice"
LET alive$ = TRUE
```

The first time you use a variable, you must introduce it with `LET`. Think of it as a handshake. After that, you can use it freely:

```
LET score$ = 0
score$ = score$ + 10
PRINT "Score: "; score$
```

BazzBasic doesn't care whether you store a number or text in the same variable. Both work:

```
LET thing$ = 42
thing$ = "forty-two"
```

That said, mixing types in the same variable tends to confuse *you* more than the interpreter, so keep things consistent. Your future self will thank you.

↑ Back to top

4.3 Constants

Some things don't change. The width of your game screen. The value of pi. The maximum number of enemies allowed on screen at once or the title of your game.

For these, use a **constant**. Constants end with `#` instead of `$`, and that `#` is your promise to both BazzBasic and yourself: *this value will not change*.

```
LET SCREEN_W# = 800
LET SCREEN_H# = 600
LET MAX_ENEMIES# = 10
LET TITLE# = "My Game"
```

Constants are declared just like variables - with `LET` - but the `#` suffix signals their purpose. By convention, write them in `UPPER_SNAKE_CASE` so they stand out clearly in your code.

Why bother? Two reasons.

First, **readability**. When you see `MAX_ENEMIES#` in your code three months later, you know exactly what it means. When you see `10`, you have to guess.

Second, **maintenance**. If your screen width appears in twenty places and you need to change it, one edit to the constant fixes everything. Twenty separate `800`s scattered across your code is a bad day waiting to happen.

BazzBasic also comes with several built-in constants ready to use:

```
PRINT PI          ' 3.14159...
PRINT KEY_ESC#    ' the Escape key value
PRINT TRUE        ' 1
PRINT FALSE       ' 0
```

↑ Back to top

4.4 Arrays

Variables and constants store one thing at a time. But what if you need to remember a whole shopping list? Or the stats for every enemy in a room? Or the letters typed by the player?

That's what **arrays** are for. An array is a named collection of values - one name, many boxes.

Declare an array with `DIM`:

```
DIM scores$
```

Then fill it using either a numeric index or a text key:

```
scores$(0) = 150
scores$(1) = 230
scores$(2) = 90
```

Or with text keys, which can make your code much more readable:

```
DIM player$
    player$("name") = "Alice"
    player$("health") = 100
    player$("level") = 3
```

Reading back from an array works just the same way:

```
PRINT player$("name")   ' Alice
PRINT player$("health") ' 100
```

One thing to keep in mind: before reading a value you're not certain exists, check with `HASKEY`:

```
IF HASKEY(player$("shield")) THEN
    PRINT "Has shield: "; player$("shield")
ELSE
    PRINT "No shield equipped."
END IF
```

Arrays in BazzBasic are flexible by nature - you can mix numeric and text keys in the same array, go as many dimensions deep as you need, and grow them at any time. There's no need to declare a size upfront.

```
DIM board$
    board$(0, 0) = "X"
    board$(0, 1) = "O"
    board$(1, 0) = "O"
```

↑ Back to top

4.5 Working with Values

Storing a value is useful. Doing something with it is where things get interesting.

The `+` operator in BazzBasic does double duty: it adds numbers *and* joins text together. The same symbol, the same idea - stick two things together.

```
LET firstName$ = "Alice"
LET lastName$ = "Smith"
LET fullName$ = firstName$ + " " + lastName$
PRINT fullName$   ' Alice Smith

LET a$ = 10
LET b$ = 5
LET result$ = a$ + b$
PRINT result$     ' 15
```

You can also bring constants into the mix. Constants are values just like any other - they just never change:

```
LET BONUS# = 50
LET score$ = 120
LET total$ = score$ + BONUS#
PRINT "Total score: "; total$   ' Total score: 170
```

This is one of the reasons constants are worth the effort. `score$ + BONUS#` reads like a sentence. `score$ + 50` leaves you guessing what that 50 is supposed to mean.

↑ Back to top

4.6 Declaring Without a Value

Sometimes you know a variable will be needed later, but you don't have a value for it yet. That is completely fine - just declare it empty:

```
LET result$
LET playerName$
LET ready$
```

This is actually good habit. Declaring all your variables at the top of the program, before anything runs, gives you a clear picture of everything your program is going to work with. It also avoids the classic mistake of using a variable you forgot to set up.

The value will be `0` for numbers and an empty string `""` for text until you assign something to it.

```
LET total$
total$ = total$ + 10   ' perfectly fine - starts at 0
PRINT total$           ' 10
```

↑ Back to top

4.7 Reading Multiple Values From an Array

Arrays really show their usefulness when you start pulling several values out together:

```
DIM player$
player$("name") = "Alice"
player$("health") = 100

PRINT player$("name") + " has " + player$("health") + " HP"
' Alice has 100 HP
```

Same idea works with numeric indexes:

```
DIM scores$
scores$(0) = 150
scores$(1) = 230

PRINT "Round 1: " + scores$(0) + "  Round 2: " + scores$(1)
' Round 1: 150  Round 2: 230
```

One thing worth knowing: if you try to read a key that doesn't exist yet, BazzBasic won't be happy about it. When in doubt, check first with `HASKEY` - you already saw that in the previous section.

↑ Back to top

4.8 Choosing the Right Tool

It's worth pausing for a second and thinking about which of the three to reach for:

- Something that **changes during the program**? variable `$`
- Something that is **set once and never changes**? constant `#`
- A **collection of related things**? array with `DIM`

Getting this right won't make your program run faster, but it will make it easier to read, easier to fix, and more enjoyable to work on. And that matters more than you might think.

↑ Back to top

5.0 Program Structure

Before we go any further, let's take a small step back and look at the bigger picture.

As your programs grow, they start to do several different things: set things up, run the main logic, maybe show some results. Keeping all of that in one big pile of code works - technically - but it quickly becomes hard to read, hard to fix, and even harder to come back to a week later.

BazzBasic uses **labels** to help you organise your code into named sections. Now let's talk about what they actually are.

A label looks like this:

```
[main]
```

It's just a name in square brackets - a landmark, a way of saying *'this is where this part of the program lives'*.

↑ Back to top

5.1 A Common Structure

Most BazzBasic programs follow a simple pattern:

```
' My guessing game
' Author: Alice

[inits]
    ' Declare all variables and constants here

[main]
    ' The actual program logic goes here

[output]
    ' Show results, if needed
END

[sub:something]
    ' More on these later
RETURN
```

Let's look at what each part is for.

`[inits]` is where you set everything up before anything runs. All your `LET` and `DIM` declarations live here. Getting into this habit early pays off - when you can see all your variables in one place, you always know what your program is working with.

`[main]` is where your program actually does its thing. Reading input, making decisions, running a loop. The real action.

`[output]` is optional - not every program needs it. But when your program collects results and shows them all at the end, it's a nice clean place to put that final block of `PRINT` statements.

`[sub:something]` We'll come back to it when we cover subroutines.

↑ Back to top

5.2 It's a Convention, Not a Rule

Here's something important: BazzBasic does not *require* you to use this structure. Labels are not mandatory. A simple one-page program might not need them at all.

```
PRINT "Hello from BazzBasic!"
```

That's a perfectly valid program.

The structure is there for *you*. As your programs grow, having clear landmarks makes them easier to navigate, easier to explain to someone else, and a lot easier to return to after a break.

Some programmers prefer different names. Some add more sections. Some drop `[output]` entirely and print things straight from `[main]`. All of that is fine. The goal is readable, understandable code - not following a rigid template.

As a starting point though, **`[inits]`**, **`[main]`**, and when needed **`[output]`** will serve you well for a long time.

We will add more sections to this structure later - as the programs we write start to grow and deserve them.

↑ Back to top

6.0 Your First Game

You now know enough to build something that actually feels like a program.

Not just printing text. Not just storing a name. An actual little game, with a question, a result, and a winner - or not.

Here is what we are going to make:

> You throw three dice. Before seeing the results, you guess which throw - the first, second, or third - will land the lowest. If you are right, you win.

Simple. But to build it, you will need almost everything you have learned so far: variables, constants, an array, INPUT, PRINT, labels - and two new things: **FOR** and **IF**.

↑ Back to top

6.1 Doing Things More Than Once with FOR

Sometimes you need to do the same thing several times. Throw a die three times. Print ten numbers. Check every item in a list.

You *could* just write the same lines over and over:

```
PRINT "Throw 1"
PRINT "Throw 2"
PRINT "Throw 3"
```

That works for three lines. But with thirty? Three hundred?

This is what **FOR** is for. It runs a block of code a set number of times, counting as it goes:

```
FOR i$ = 1 TO 3
    PRINT "Throw " + i$
NEXT
```

Read it like this: *start with i$ at 1, run the block, add 1 to i$, repeat - stop after 3.*

`i$` is just a variable. It gets the current count on every pass, which means you can use it inside the loop:

```
FOR i$ = 1 TO 3
    PRINT "Throw " + i$ + ": " + (RND(6) + 1)
NEXT
```

Each time through, `i$` is a different number - 1, then 2, then 3 - and the loop uses it to label the throw.

`NEXT` marks the end of the loop. Everything between `FOR` and `NEXT` repeats.

By default, the counter goes up by one each time. If you need a different step, `STEP` lets you control it:

```
FOR i$ = 10 TO 1 STEP -1
    PRINT i$
NEXT
```

That counts down from 10 to 1. You can use any step size - positive or negative.

FOR loops and arrays are natural partners. The loop gives you an index, the array holds the values:

```
DIM throws$
FOR i$ = 1 TO 3
    throws$(i$) = RND(6) + 1
NEXT
```

Three dice thrown, three results stored, four lines of code. That is the idea.

↑ Back to top

6.2 Making Decisions with IF

Programs need to make choices. Is the player's guess correct? Is the score high enough? Is the game over?

In BazzBasic, you make choices with `IF`:

```
IF score$ > 100 THEN
    PRINT "You win!"
END IF
```

Read it like a sentence: *if score is greater than 100, then print "You win".*

If the condition is false, BazzBasic simply skips everything between `IF` and `END IF` and moves on.

You can also handle the opposite case with `ELSE`:

```
IF score$ > 100 THEN
    PRINT "You win!"
ELSE
    PRINT "Not quite."
END IF
```

And if you need to check several things in a row, `ELSEIF` keeps the chain going:

```
IF score$ > 100 THEN
    PRINT "You win!"
ELSEIF score$ = 100 THEN
    PRINT "So close!"
ELSE
    PRINT "Not quite."
END IF
```

Only one branch ever runs - BazzBasic checks from the top and stops at the first condition that is true.

For quick, simple checks where you only need one line, you can also write everything on a single line:

```
IF lives$ = 0 THEN PRINT "Game over."
```

That is all you need to know about IF for now. Let's use it.

↑ Back to top

6.3 The Game

Here is the complete program. Read through it once before we walk through it together.

```
' ============================================
' Dice Guessing Game
' Guess which throw lands lowest!
' ============================================

[inits]
    LET MAX_THROWS# = 3
    LET guessedThrow$
    LET lowestValue$
    LET lowestThrow$
    LET i$

    DIM throws$

[main]
    PRINT "Three dice will be thrown."
    PRINT "Guess which throw (1, 2, or 3) will be the lowest."
    PRINT ""
    INPUT "Your guess: ", guessedThrow$

    PRINT ""
    PRINT "Throwing dice..."
    PRINT ""

    FOR i$ = 1 TO MAX_THROWS#
        throws$(i$) = RND(6) + 1
        PRINT "Throw " + i$ + ": " + throws$(i$)
    NEXT

[output]
    lowestValue$ = throws$(1)
    lowestThrow$ = 1

    FOR i$ = 2 TO MAX_THROWS#
        IF throws$(i$) < lowestValue$ THEN
            lowestValue$ = throws$(i$)
            lowestThrow$ = i$
        END IF
    NEXT

    PRINT ""
    PRINT "The lowest throw was throw " + lowestThrow$ + " with a " + lowestValue$ + "."
    PRINT ""

    IF guessedThrow$ = lowestThrow$ THEN
        PRINT "You guessed right. Well done!"
    ELSE
        PRINT "Bad luck. You guessed throw " + guessedThrow$ + "."
    END IF

    PRINT "Press any key to end."
    LET i$ = WAITKEY()
END
```

↑ Back to top

6.4 Walking Through It

Let's look at what each part does.

`[inits]` declares everything the program will need - including the constant `MAX_THROWS#` . Constants belong here too, right alongside your variables. Keeping all declarations in one place means you always know what your program is working with before a single line of logic runs. The array `throws$` will hold the dice results, and the other variables store the player's guess, the lowest value found, and which throw it came from.

`[main]` runs the game itself. It asks for a guess with `INPUT` , then uses a `FOR` loop to throw the dice. `RND(6) + 1` gives a random number from 1 to 6, just like a

real die. Each result goes into the array using the throw number as the index, and gets printed straight away so the player can see the results unfold.

`[output]` figures out the winner. It starts by assuming the first throw is the lowest - that is always a safe starting point. Then it loops through the remaining throws, and each time it finds a smaller value it updates both `lowestValue$` and `lowestThrow$`. By the time the loop ends, it has found the answer.

Finally, an `IF` tells the player whether they guessed correctly. `WAITKEY()` holds the console open so the result can be read before the window closes.

↑ Back to top

6.5 Try It Yourself

Run the program a few times and watch what happens.

Then try making a small change: what if you add a `PRINT` inside the loop in `[output]` to show each throw being compared? Or what if you congratulate the player differently when there is a tie - two throws with the same lowest value?

You do not have to. But poking at working code is one of the best ways to learn.

Congratulations.

You have just built:

1. A complete game with input, logic, and output
2. Your first use of FOR to repeat work without repeating code
3. Your first use of IF to make a decision
4. A loop that searches through an array for the best answer

That is more than it might seem. This same structure - collect data, loop to find something, then report - appears in programs of every size and kind.

↑ Back to top

7.0 Reality Check

↑ Back to top

7.1 A Word From Author

I am writing this chapter because I want to tell you honestly what programming is.

You have probably seen how in some movie the hero - or at least a nerd in a significant supporting role - writes a virus that infects an alien invasion fleet's systems - while telling a sad story about a summer camp accident. In 35 seconds.

The hero's fingers move across the keys faster than a high-speed camera can follow.

This is not reality.

The previous example - rolling three dice and guessing which one was lowest - required 53 lines of code.

And when you start adding logic, options, and different outputs, the size of a program grows very quickly.

For example, Boxing.bas, which is a relatively simple boxing simulator without any graphics, requires 450 lines of code.

Don't take this as a reason to stop, but as a challenge to learn.

If you haven't planned and organised your code before you start, you will be in trouble very soon. And unfortunately, that is one of the most common things that breaks a beginner's momentum - and quietly kills a good hobby before it has had a chance to grow.

When you start planning a program, think about what it needs to do:

1. What is its purpose?
2. What does it need to know?
3. What does it need to show?
4. What logic does it need to get there?

Once you have those answers, the right variables, constants, and arrays will start suggesting themselves naturally.

↑ Back to top

8.0 More Loops

You have already used a `FOR` loop in chapter 6. It did exactly what you needed: it ran the same block of code a set number of times and kept count.

But not every problem fits that shape.

Sometimes you do not know in advance how many times something needs to happen. You want to keep going until the player gets the answer right, or until they choose to quit, or until some condition in your program changes. Counting does not help you there - you need something that just keeps watching and waiting.

That is what this chapter is about: understanding both kinds of loop properly, and knowing which one to reach for.

↑ Back to top

8.1 WHILE

`WHILE` is the simpler of the two to describe. It checks a condition, and as long as that condition is true, it runs the block of code inside it. When the condition becomes false, it stops.

```
WHILE lives$ > 0
    PRINT "Still in the game!"
    lives$ = lives$ - 1
WEND
```

`WEND` marks the end of the loop - short for *while end*. Everything between `WHILE` and `WEND` repeats.

Read it like a sentence: *while lives are greater than zero, keep going.*

The important thing to understand is that `WHILE` checks the condition **at the top**, before each run of the loop. If the condition is already false when the program first reaches the `WHILE`, the loop body never runs at all.

```
LET lives$ = 0
WHILE lives$ > 0
    PRINT "This never prints."
WEND
```

That is not a bug - it is the intended behaviour. The loop checked, found the condition false, and skipped the whole block.

A very common use of `WHILE` is keeping a program running until the user decides to stop:

```
LET answer$
WHILE answer$ <> "quit"
    INPUT "Type something (or 'quit' to stop): ", answer$
    IF answer$ <> "quit" THEN PRINT "You typed: " + answer$
WEND
PRINT "Goodbye."
```

Notice that `answer$` needs to be declared before the loop. The `WHILE` checks it immediately on the first pass, so it must exist. An empty string `""` is not equal to `"quit"`, which means the loop starts running - exactly as intended.

`WHILE` can also loop forever deliberately, when you want the program to keep running until something inside the loop triggers an exit:

```
WHILE TRUE
    INPUT "Guess the number (1-10): ", guess$
    IF guess$ = 7 THEN
        PRINT "Correct!"
        END
    END IF
    PRINT "Wrong, try again."
WEND
```

`WHILE TRUE` never stops on its own - the condition is always true. Here, `END` exits the program when the player guesses correctly. You will see this pattern often in game loops and menu systems.

↑ Back to top

8.2 FOR

You have already met `FOR`. Let's look at it properly now.

`FOR` runs a block of code a specific number of times, counting with a variable as it goes:

```
FOR i$ = 1 TO 5
    PRINT i$
NEXT
```

Output: 1, 2, 3, 4, 5. One per line. The counter `i$` starts at 1, the block runs, then `i$` increases by 1, and the process repeats until `i$` would exceed 5.

`NEXT` marks the end of the loop body.

STEP

By default, the counter increases by 1 each time. `STEP` lets you change that:

```
FOR i$ = 0 TO 10 STEP 2
    PRINT i$
NEXT
```

Output: 0, 2, 4, 6, 8, 10. Every even number.

`STEP` works with any value - including decimal-values:

```
FOR i$ = 0 TO 1 STEP 0.25
    PRINT i$
NEXT
```

Output: 0, 0.25, 0.5, 0.75, 1.

Counting backwards

`STEP` also accepts negative values, which lets you count down:

```
FOR i$ = 5 TO 1 STEP -1
    PRINT i$
NEXT
```

Output: 5, 4, 3, 2, 1.

One thing to watch: if you count downwards, the start must be *larger* than the end. If you accidentally write `FOR i$ = 1 TO 5 STEP -1`, the condition is already false on the first check and the loop never runs - just like a `WHILE` with a false starting condition.

The counter is a real variable

Because the counter is an ordinary variable, you can use it inside the loop for more than just labelling:

```
FOR i$ = 1 TO 5
    PRINT i$ + ": " + (i$ * i$)
NEXT
```

Output: 1: 1, 2: 4, 3: 9, 4: 16, 5: 25. Squares of 1 through 5.

And just like any variable, `i$` keeps its last value after the loop finishes. Usually you do not need that, but it is worth knowing.

↑ Back to top

8.2.1 Exception with FOR variable declaration

BazzBasic generally demands, that variables must be declared with LET before they can be used.
With FOR, there is an exception: FOR can declare the variable itself. You do not need to.

↑ Back to top

8.3 When to Use Which

This is one of those questions that gets easier with practice, but a simple rule covers most cases:

You know how many times - use `FOR`. You are waiting for something to change - use `WHILE`.

When you want to do something with every item in a list, process a fixed number of rounds, or count through a range of numbers: `FOR`. The number of repetitions is decided before the loop starts.

When you want to keep asking until the player answers correctly, keep a game running until the player quits, or wait until some value in your program reaches a certain point: `WHILE`. The loop itself does not know how many times it will run - the condition decides.

In practice, you will almost never be confused between the two. The problem itself usually makes it obvious which one fits.

One last thing worth noting: `FOR` and `WHILE` can be nested - a loop inside a loop. You already did this in chapter 6 without naming it. It is perfectly natural, and we will use it again in the chapters ahead.

```
FOR row$ = 1 TO 3
    FOR col$ = 1 TO 3
        PRINT row$ + "," + col$ + "  ";
    NEXT
    PRINT ""
NEXT
```

The inner loop completes fully for every single step of the outer loop. Three rows, three columns each: nine combinations total.

↑ Back to top

9.0 IF Conditions

You have already used `IF` in chapter 6. It made a simple decision: was the player's guess correct or not?

That was enough for the moment. But `IF` can do considerably more than that, and now that you are writing programs with loops and more moving parts, it is worth understanding it properly.

↑ Back to top

9.1 IF - THEN - END IF

The basic form you already know:

```
IF score$ >= 100 THEN
    PRINT "You win!"
END IF
```

BazzBasic evaluates the condition between `IF` and `THEN`. If it is true, everything inside the block runs. If it is false, the whole block is skipped and the program continues after `END IF`.

`ENDIF` also works as a single word - both are accepted.

↑ Back to top

9.1.1 Comparison Operators

These are the tools you use to build conditions:

Operator	Meaning
`=`	Equal to
`<>`	Not equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to

Operator	Meaning

Nothing surprising there. The one worth highlighting is `<>` for *not equal* - it trips up beginners who expect `!=` from other languages.

AND and OR

You can combine conditions in a single `IF` using `AND` and `OR`:

```
IF age$ >= 18 AND hasTicket$ = TRUE THEN
    PRINT "Welcome in."
END IF
```

```
IF answer$ = "yes" OR answer$ = "y" THEN
    PRINT "Confirmed."
END IF
```

`AND` requires both conditions to be true. `OR` requires at least one. You can chain more than two, but keep it readable - if a condition is getting long and tangled, that is usually a sign it wants to be broken apart.

↑ Back to top

9.2 IF - THEN - ELSEIF - ELSE - END IF

When you have more than two possible outcomes, `ELSEIF` lets you chain conditions together:

```
IF score$ >= 90 THEN
    PRINT "Grade: A"
ELSEIF score$ >= 80 THEN
    PRINT "Grade: B"
ELSEIF score$ >= 70 THEN
    PRINT "Grade: C"
ELSE
    PRINT "Grade: F"
END IF
```

BazzBasic checks from the top down. The moment it finds a condition that is true, it runs that block and skips everything else - including the remaining `ELSEIF` branches and the `ELSE`. Only one branch ever runs.

`ELSE` at the end is optional. It is the fallback - it runs if none of the conditions above were true. If you leave it out and nothing matches, the whole structure is simply skipped.

↑ Back to top

9.3 Nested IF

You can place an `IF` inside another `IF`. This is called nesting, and it is completely natural:

```
IF loggedIn$ = TRUE THEN
    IF isAdmin$ = TRUE THEN
        PRINT "Welcome, administrator."
    ELSE
        PRINT "Welcome, user."
    END IF
ELSE
    PRINT "Please log in first."
END IF
```

Each `IF` has its own `END IF`. BazzBasic matches them from the inside out, so they always pair up correctly as long as you close each one.

Nesting works, but be careful not to go too many levels deep. Three levels in, the logic starts becoming hard to follow - for you as much as anyone else. If you find yourself deeply nested, it is often a sign the logic can be reorganised.

↑ Back to top

9.4 One-liner form

For short, simple checks, you can write everything on a single line:

```
IF lives$ = 0 THEN PRINT "Game over."
```

This is the same as the block form - just compressed. It is handy for quick checks, but only works when the action is a single statement. The moment you need two things to happen, switch back to the block form with `END IF`.

You can also include a single `ELSE` on the same line:

```
IF door$ = "open" THEN PRINT "You walk through." ELSE PRINT "The door is locked."
```

Again, fine for simple cases. If it starts feeling crowded, expand it into a block.

↑ Back to top

9.5 TRUE and FALSE

BazzBasic has built-in constants `TRUE` and `FALSE`. Under the hood, `TRUE` is `1` and `FALSE` is `0`, which means comparisons like these both work:

```
IF running$ = TRUE THEN ...
IF running$ THEN ...
```

Both are identical. The second form - checking the variable directly without comparing it to `TRUE` - is a common shorthand you will see often.

Similarly, `IF NOT running$ THEN` is equivalent to `IF running$ = FALSE THEN`. Either style is fine.

↑ Back to top

10.0 Controlling The Screen

So far your programs have scrolled text down the console one line at a time. That works fine for simple output, but it has limits. A scoreboard that rewrites itself in place feels very different from one that just keeps printing new lines. A menu with colored options feels very different from plain white text on black.

This chapter gives you proper control over the console: where text appears, what color it is, and how the screen is managed. You have already used `PRINT` and `INPUT` throughout the guide - here you will learn what they can actually do.

↑ Back to top

10.1 PRINT

You know the basics. Time for the rest of it.

`PRINT` separates values with a **semicolon** or a **comma**. They behave differently:

```
PRINT "Hello"; " "; "World"     ' Hello World  (no extra space)
PRINT "Score:"; 100             ' Score:100
PRINT "Name:", "Alice"          ' Name:        Alice  (tab stop)
```

The semicolon joins things with no space between them. The comma advances to the next tab position - useful for simple columns.

By default, `PRINT` always moves to a new line when it finishes. A **trailing semicolon** suppresses that:

```
PRINT "Loading";
PRINT "...";
PRINT " Done."
```

Output: `Loading... Done.` - all on one line, because each `PRINT` picked up exactly where the last one stopped.

This becomes useful when you want to build output piece by piece without jumping to a new line each time.

↑ Back to top

10.1.1 Escape sequences

These sequences live inside strings and give you a few extra tools:

Sequence	Effect
`\n`	New line
`\t`	Tab
`\"`	Literal quote mark
`\\`	Literal backslash

```
PRINT "Name:\tAlice\nScore:\t100"
```

Output:

```
Name:   Alice
Score:  100
```

↑ Back to top

10.2 INPUT and LINE INPUT

`INPUT` has one behaviour worth knowing more clearly: it splits the user's response on **whitespace and commas**. That means it can read several values from a single line:

```
INPUT "Enter two numbers: ", first$, second$
PRINT "Sum: " + (first$ + second$)
```

The user types `10 20` or `10,20` and both values land in their own variables. Handy for quick data entry.

The drawback is that a response like `Alice Smith` would split at the space, putting `Alice` in one variable and `Smith` in the next - or being lost if there is only one variable waiting.

For anything where the user might type spaces, use `LINE INPUT` instead:

```
LINE INPUT "Enter your full name: ", fullName$
PRINT "Hello, " + fullName$
```

`LINE INPUT` reads the entire line as-is, spaces and all, into a single variable.

↑ Back to top

10.3 CLS

`CLS` clears the entire console screen:

```
PRINT "This will disappear."
SLEEP 1000
CLS
PRINT "Fresh start."
```

Simple and blunt. Everything that was on screen is gone, the cursor moves back to the top-left corner, and output continues from there.

Use `CLS` when you want a completely clean slate - between menu screens, at the start of a new round, or whenever old output would just get in the way.

↑ Back to top

10.4 LOCATE

`CLS` clears everything. `LOCATE` is more surgical - it moves the cursor to a specific position without touching anything else:

```
LOCATE 5, 10
PRINT "Hello from row 5, column 10"
```

The first number is the **row**, the second is the **column**. Both are 1-based - row 1, column 1 is the top-left corner of the screen.

This lets you write to any part of the screen at any time, regardless of where the cursor currently is:

```
LOCATE 1, 1
PRINT "Score: 0   "

' ... some game logic ...

LOCATE 1, 8
PRINT score$
```

The score updates in place at row 1, column 8 - without scrolling, without clearing, without disturbing anything else on screen. That is the real power of `LOCATE`.

A few things worth knowing:

- If you print past the end of a line, the text wraps normally
- Writing over existing text replaces it character by character - old characters that are wider than the new text will show through, so pad with spaces if needed:

```
LOCATE 3, 1
PRINT "Score: " + score$ + "   "    ' trailing spaces clear leftover digits
```

↑ Back to top

10.5 CURPOS

`CURPOS` reads where the cursor currently is - the opposite of `LOCATE`:

```
LET currentRow$ = CURPOS("row")
LET currentCol$ = CURPOS("col")
```

This is useful when you need to save a position, do something elsewhere on screen, and then return:

```
LET savedRow$ = CURPOS("row")
LET savedCol$ = CURPOS("col")

LOCATE 1, 1
PRINT "Status: working..."

LOCATE savedRow$, savedCol$
' continue from where we were
```

On its own `CURPOS` is a small tool. Combined with `LOCATE` it gives you proper cursor management.

↑ Back to top

10.6 COLOR

`COLOR` sets the foreground and background colors for everything printed after it:

```
COLOR 14, 1
PRINT "Yellow text on blue background"
```

The first number is the **foreground** (text color), the second is the **background**. BazzBasic uses a 16-color palette:

Number	Color	Number	Color
0	Black	8	Dark Gray

Number	Color	Number	Color
1	Blue	9	Light Blue
2	Green	10	Light Green
3	Cyan	11	Light Cyan
4	Red	12	Light Red
5	Magenta	13	Light Magenta
6	Brown	14	Yellow
7	Light Gray	15	White

`COLOR` stays in effect until you change it again. To return to normal:

```
COLOR 7, 0    ' light gray on black - the default console look
```

↑ Back to top

10.7 A Practical Example

Here is a small program that puts these tools together - a simple status display that updates in place:

```
[inits]
    LET score$   = 0
    LET lives$   = 3
    LET counter$ = 0

[main]
    CLS
    WHILE lives$ > 0
        ' Draw header once, update values in place
        COLOR 11, 0
        LOCATE 1, 1 : PRINT "=== GAME STATUS ==="
        COLOR 7, 0
        LOCATE 2, 1 : PRINT "Score: "
        LOCATE 3, 1 : PRINT "Lives: "

        ' Update values without redrawing labels
        COLOR 14, 0
        LOCATE 2, 8 : PRINT score$ + "   "
        LOCATE 3, 8 : PRINT lives$ + " "

        ' Simulate something happening
        score$   = score$ + 10
        counter$ = counter$ + 1
        IF counter$ = 5 THEN
            lives$   = lives$ - 1
            counter$ = 0
        END IF

        SLEEP 300
    WEND

    COLOR 12, 0
    LOCATE 5, 1
    PRINT "Game over!"
    COLOR 7, 0

[end-game]
    LET a$ = WAITKEY()
END
```

The labels stay still. Only the numbers change. The screen never flickers. That is the difference between a program that just prints and one that actually controls what the user sees.

You now have full control over the console. Where text appears, what color it is, and how you manage what the screen shows at any given moment - all of it is in your hands.

↑ Back to top

11.0 Controlling The Program

You have been using labels since chapter 5. `[inits]`, `[main]`, `[output]` - they have been there all along, organising your code into named sections.

Until now, though, the program has simply fallen through them one after another from top to bottom. Labels were landmarks, not destinations.

That changes here. `GOSUB` and `GOTO` let you jump to a label deliberately - and in the case of `GOSUB`, come back again when the work is done.

↑ Back to top

11.1 GOSUB

`GOSUB` jumps to a label, runs the code there, and returns to exactly where it left off when it hits a `RETURN`.

```
GOSUB [sub:greet]
PRINT "Back in main."
END

[sub:greet]
    PRINT "Hello!"
RETURN
```

Output:

```
Hello!
Back in main.
```

Read it as task delegation: *go handle the greeting, then come back*. The `END` before `[sub:greet]` is important - without it, the program would fall into the subroutine a second time after `GOSUB` returns.

↑ Back to top

11.1.1 Why GOSUB Matters

As programs grow, the same block of logic tends to appear in multiple places. Without *GOSUB* you copy and paste it - and then maintain three copies when something needs to change. With *GOSUB* you write it once and call it whenever needed:

```
[inits]
    LET score$  = 0
    LET lives$  = 3
    LET choice$

[main]
    GOSUB [sub:drawScreen]
    WHILE lives$ > 0
        INPUT "Your move: ", choice$
        IF choice$ = "q" THEN lives$ = 0
        score$ = score$ + 10
        GOSUB [sub:drawScreen]
    WEND
    GOSUB [sub:gameOver]
END

[sub:drawScreen]
    CLS
    COLOR 11, 0 : LOCATE 1, 1 : PRINT "=== MY GAME ==="
    COLOR 7, 0
    LOCATE 2, 1 : PRINT "Score: " + score$
    LOCATE 3, 1 : PRINT "Lives: " + lives$
RETURN

[sub:gameOver]
    COLOR 12, 0
    LOCATE 5, 1
    PRINT "Game over! Final score: " + score$
    COLOR 7, 0
    LET choice$ = WAITKEY()
RETURN
```

The drawing logic lives in one place. Call it as many times as you like. Change it once and it changes everywhere.

↑ Back to top

11.1.2 GOSUB And Program Structure

You already know the convention of naming subroutine labels `[sub:name]`. That is not a rule BazzBasic enforces - it is a habit worth keeping. It makes subroutines immediately recognisable when reading code, and separates them clearly from from section labels and section markers.

Keep `END` before your subroutines. Everything between the last line of your main logic and `END` should be subroutines only - code that only runs when called, never by accident.

↑ Back to top

11.2 GOTO

`GOTO` jumps to a label without any intention of coming back. There is no `RETURN`. Execution simply continues from the new location.

```
GOTO [main]

[main]
    PRINT "Jumped here."
END
```

BazzBasic supports `GOTO` fully, and you will encounter it in older BASIC code and in some examples around the web. It has its place in the history of the language.

In practice, though, `GOTO` is rarely the right tool. A jump with no return makes programs hard to follow - you lose track of where execution came from and where it is going. `WHILE` handles looping. `GOSUB` handles reusable blocks. Between the two of them, most things `GOTO` might solve are better solved another way.

There is one case where `GOTO` appears naturally in BazzBasic - jumping to a label from a one-liner `IF`:

```
IF lives$ = 0 THEN GOTO [game_over]
```

That is readable and clear. For short, obvious jumps like this, it is acceptable. For anything more complex, prefer a `WHILE` loop or a `GOSUB`.

That is the full picture of program flow control. `GOSUB` is your main tool for keeping code organised and reusable. `GOTO` is there when you need it, but rarely the first answer.

↑ Back to top

12.0 Putting It All Together

You now have a solid set of tools. Variables and constants. Arrays. Loops - both `FOR` and `WHILE`. Conditions with `IF`. Screen control with `LOCATE`, `COLOR`, and `CLS`. And program structure with `GOSUB`.

Before we look at the game, one small addition is needed.

↑ Back to top

12.1 A Quick Look at WAITKEY

You have used `INPUT` to read text from the user. But sometimes you just want a single keypress - no typing, no Enter. That is what `WAITKEY` does.

```
LET key$ = WAITKEY()
```

The program stops and waits. The moment the user presses any key, execution continues and the key value lands in `key$`.

You can also tell `WAITKEY` to only accept specific keys:

```
LET key$ = WAITKEY(KEY_1#, KEY_2#)
```

Now the program waits specifically for 1 or 2. Any other key is ignored.

Keyboard input has more to it than this - `INKEY`, `KEYDOWN`, key constants - and we will cover all of it properly later. For now, `WAITKEY` is all the game needs.

↑ Back to top

12.2 Russian Roulette

This game originally appeared in the book *BASIC Computer Games* published by Creative Computing in 1978. It is one of the oldest and simplest interactive programs ever written for home computers - and it makes a perfect example for everything you have learned.

The game is short, the logic is clear, and every tool from the last several chapters shows up naturally:

- `[inits]` with constants and variables
- `WHILE` loops - nested, with two flags controlling the flow
- `IF` for decisions
- `GOSUB` for clean, reusable sections
- `CLS`, `COLOR`, `LOCATE` for screen control
- `WAITKEY` for input

Read through the full program first. Then we will walk through it together.

```
' ==========================================
' Russian Roulette
' Originally from CREATIVE COMPUTING
' Morristown, New Jersey
'
' BazzBasic version
' https://github.com/EkBass/BazzBasic/blob/main/Examples/RussianRoulette.bas
' ==========================================

[inits]
    LET BLACK#   = 0
    LET RED#     = 4
    LET CYAN#    = 11
    LET WHITE#   = 15
    LET playing$ = TRUE
    LET alive$   = TRUE
```

```
    LET choice$

[main]
    WHILE playing$
        GOSUB [sub:title]
        alive$ = TRUE

        WHILE playing$ AND alive$
            GOSUB [sub:menu]
            IF playing$ THEN GOSUB [sub:shoot]
        WEND
    WEND

    PRINT " Smart move.\n"
END

[sub:title]
    COLOR CYAN#, BLACK#
    CLS
    PRINT " "; REPEAT("*", 21)
    PRINT " *"; REPEAT(" ", 19); "*"
    PRINT " *  RUSSIAN ROULETTE  *"
    PRINT " *"; REPEAT(" ", 19); "*"
    PRINT " "; REPEAT("*", 21)
    PRINT "\n CREATIVE COMPUTING\n MORRISTOWN, NEW JERSEY\n"
    COLOR WHITE#, BLACK#
RETURN

[sub:menu]
    PRINT " HERE IS A REVOLVER."
    PRINT " 1 - Spin the chamber and pull the trigger."
    PRINT " 2 - Walk away.\n"

    choice$ = WAITKEY(KEY_1#, KEY_2#)
    PRINT " "; CHR(choice$); "\n"

    IF choice$ = KEY_2# THEN playing$ = FALSE
RETURN

[sub:shoot]
    IF RND(6) = 0 THEN
        COLOR RED#, BLACK#
        PRINT " *** BANG! ***\n"
        COLOR WHITE#, BLACK#
        PRINT " You're dead."
        PRINT " Condolences will be sent to your next of kin.\n"
        SLEEP 3000
        alive$ = FALSE
        RETURN
    END IF

    PRINT " *click*"
    PRINT " You survived - this time.\n"
    SLEEP 1500
RETURN
```

↑ Back to top

12.3 Walking Through It

`[inits]` declares everything up front. The color constants keep the numbers readable throughout - `RED#` means something, `4` does not. The two flags `playing$` and `alive$` are the engine that drives the whole game.

`[main]` is two nested `WHILE` loops. The outer one runs as long as `playing$` is true - meaning the player has not chosen to walk away. Each pass through it shows the title screen and resets `alive$` to TRUE before starting a new round.

The inner loop runs as long as both `playing$ AND alive$` are true - meaning the player is still in the game and still breathing. Each pass shows the menu and, if the player chose to shoot, fires the gun.

The `IF playing$ THEN GOSUB [sub:shoot]` guard is a small but important detail. When the player presses 2 in `sub:menu`, `playing$` becomes FALSE immediately. Without the guard, the program would go on to fire the gun anyway before the loop condition gets checked. The guard prevents that.

`[sub:title]` draws the title screen with `CLS` and `COLOR`, then hands control back. Nothing complicated - but because it is a subroutine, the main loop can call it cleanly at the start of every new game without repeating code.

`[sub:menu]` presents the choice and waits. `WAITKEY(KEY_1#, KEY_2#)` blocks until exactly one of those two keys is pressed - no Enter required, no other key accepted. If the player walks away, `playing$ = FALSE` is set here and the loop exits naturally on the next condition check.

`[sub:shoot]` is where the tension lives. `RND(6) = 0` gives a one-in-six chance - six possible values (0 through 5), one of them fatal. If that value comes up, the screen turns red, the player is told they are dead, `alive$` is set to FALSE, and the subroutine returns. The inner loop exits, the outer loop shows the title again, and a new game begins.

If the shot is not fatal, a quiet `*click*` is printed, a short pause lets the moment breathe, and the subroutine returns normally - straight back into the inner loop for another round at the menu.

Congratulations.

You have just read and understood a complete game. Every line of it uses tools you already have. No magic, no tricks - just variables, loops, conditions, subroutines, and screen control working together.

That is all programming is, at any scale.

↑ Back to top

13.0 User-Defined Functions

You already know `GOSUB`. It jumps to a subroutine, does some work, and comes back. That is useful - but subroutines share everything with the rest of the program. Every variable, every array. There is no boundary between them and the code that calls them.

Sometimes that shared access is exactly what you want. But sometimes you want something smaller and more self-contained. A piece of logic with its own inputs, its own working space, and a single value to hand back when it is done.

That is what `DEF FN` gives you.

↑ Back to top

13.1 Defining and Calling

A function is defined with `DEF FN` and closed with `END DEF`. The name follows the same rules as any variable - it ends with `$` just like any variable:

```
DEF FN Double$(n$)
    RETURN n$ * 2
END DEF
```

To call it, use the `FN` keyword:

```
PRINT FN Double$(5)     ' prints 10
```

Functions must be defined **before** they are called. The safest habit is to put all your function definitions at the top of the file, before `[inits]`.

↑ Back to top

13.2 Parameters

Parameters are the values you pass into a function. They are declared in the parentheses after the function name and behave like local variables - they only exist inside the function:

```
DEF FN Add$(a$, b$)
    RETURN a$ + b$
END DEF

PRINT FN Add$(10, 5)    ' 15
PRINT FN Add$(3, 7)     ' 10
```

You can pass as many parameters as you need. A function with no parameters uses empty parentheses:

```
DEF FN Greeting$()
    RETURN "Hello!"
END DEF

PRINT FN Greeting$()
```

Parameters are passed **by value** - the function gets a copy of the value, not the original. Changing a parameter inside a function has no effect on the variable that was passed in:

```
DEF FN TryToChange$(x$)
    x$ = 999
    RETURN x$
END DEF

[inits]
    LET number$ = 42

[main]
    PRINT FN TryToChange$(number$)  ' 999
    PRINT number$                    ' still 42 - unchanged
```

↑ Back to top

13.3 Scope

This is the most important thing to understand about functions in BazzBasic.

A function is completely isolated. It cannot see the variables declared in your main program, and your main program cannot see the variables declared inside the function. They live in separate spaces.

```
DEF FN Example$()
    RETURN score$    ' ERROR - score$ does not exist here
END DEF

[inits]
    LET score$ = 100
```

The only things a function can see from outside itself are **global constants** - the ones ending in `#`:

```
LET MAX_SCORE# = 1000

DEF FN IsWinner$(score$)
    IF score$ >= MAX_SCORE# THEN RETURN TRUE
    RETURN FALSE
END DEF
```

`MAX_SCORE#` is visible inside the function. A variable `maxScore$` would not be.

This isolation is a feature, not a limitation. It means you can write a function and trust that it will behave the same way no matter where it is called from. Nothing outside can accidentally interfere with what is happening inside.

↑ Back to top

13.4 The Return Value Rule

Every function call produces a value. That value **must** be used. Calling a function and ignoring what it returns is an error:

```
FN Double$(5)           ' ERROR - return value unused
LET result$ = FN Double$(5)   ' correct
PRINT FN Double$(5)           ' correct
```

If you need to call a function for what it does rather than what it returns, capture it in a temporary variable:

```
LET temp$ = FN DoSomething$()
```

This rule exists to keep things honest. If a function returns a value, something should be done with it.

↑ Back to top

13.5 Recursion

A function can call itself. This is called **recursion**, and it is one of the more elegant ideas in programming.

The classic example is calculating a factorial - the product of all integers from 1 to n:

```
DEF FN Factorial$(n$)
    IF n$ <= 1 THEN RETURN 1
    RETURN n$ * FN Factorial$(n$ - 1)
END DEF

[main]
    PRINT FN Factorial$(5)   ' 120
```

Read the recursive case out loud: *the factorial of n is n multiplied by the factorial of n minus 1*. That is a true mathematical statement, and the code says exactly the same thing.

Every recursive function needs a **base case** - the condition where it stops calling itself and returns a plain value. Here that is `IF n$ <= 1 THEN RETURN 1`. Without a base case, the function would call itself forever.

A real example from the BazzBasic examples library shows recursion applied to the Fibonacci sequence - three different approaches to the same problem, side by side:

```
DEF FN FibRecursive$(n$)
    IF n$ <= 1 THEN RETURN n$
    RETURN FN FibRecursive$(n$ - 1) + FN FibRecursive$(n$ - 2)
END DEF

DEF FN FibIterative$(n$)
    IF n$ <= 1 THEN RETURN n$
    LET a$ = 0
    LET b$ = 1
    FOR i$ = 2 TO n$
        LET temp$ = a$ + b$
        a$ = b$
        b$ = temp$
    NEXT
    RETURN b$
END DEF

[main]
    INPUT "Enter n: ", n$
    PRINT "Recursive: "; FN FibRecursive$(n$)
    PRINT "Iterative: "; FN FibIterative$(n$)
END
```

Both give the same answer. The recursive version reads almost like the mathematical definition of Fibonacci. The iterative version is faster for large numbers because it does not build up a long chain of nested calls. Neither is wrong - they are different tools for the same job.

↑ Back to top

13.6 GOSUB vs DEF FN

Now that you have both, a simple guide for choosing between them:

Use `GOSUB` when the code needs to work with your program's variables directly - drawing the screen, updating game state, printing a report. Subroutines are part of the program, sharing its world.

Use `DEF FN` when you have a calculation or transformation that takes some inputs and produces a result. Functions are self-contained. They do not need to know anything about the program around them.

A good function could be lifted out of one program and dropped into another without changing a single line. A good subroutine is usually woven into the specific program it belongs to.

↑ Back to top

14.0 User Input

You have already used `INPUT` and got a brief introduction to `WAITKEY` in chapter 12. But there is a whole other side to user input that `INPUT` cannot handle at all.

`INPUT` waits patiently for the user to type something and press Enter. That is fine for questions and menus. It is useless for a game where the player needs to move a character, or for any program that needs to react the moment a key is pressed - not after Enter.

BazzBasic gives you several tools for different situations. This chapter covers all of them.

↑ Back to top

14.1 WAITKEY

You have already seen this one. `WAITKEY` stops the program and waits for a keypress. The moment a key is pressed, execution continues and the key value is returned:

```
LET key$ = WAITKEY()    ' wait for any key
```

You can restrict it to specific keys by listing them:

```
LET key$ = WAITKEY(KEY_1#, KEY_2#, KEY_3#)
```

Only those keys will be accepted. Anything else is silently ignored - the program keeps waiting.

`WAITKEY` is the right tool for menus, title screens, and "press any key to continue" moments. The program has nothing to do until the user responds, so blocking is exactly what you want.

↑ Back to top

14.2 INKEY

`INKEY` is the non-blocking alternative. It checks whether a key is currently being pressed and returns its value immediately - or returns 0 if nothing is pressed:

```
LET key$ = INKEY
IF key$ = KEY_ESC# THEN END
```

The crucial difference from `WAITKEY`: `INKEY` never waits. It checks and moves on. That makes it the right tool for game loops, where the program has work to do every frame regardless of whether the user is pressing anything:

```
[main]
    WHILE TRUE
        LET key$ = INKEY
        IF key$ = KEY_ESC# THEN END
        IF key$ = KEY_LEFT# THEN position$ = position$ - 1
        IF key$ = KEY_RIGHT# THEN position$ = position$ + 1
        GOSUB [sub:draw]
        SLEEP 16
    WEND
```

Each pass through the loop takes a snapshot of the keyboard at that instant. Pressed - act on it. Not pressed - continue anyway.

↑ Back to top

14.3 KEYDOWN

`KEYDOWN` checks whether a specific key is **held down** at the moment of the call:

```
IF KEYDOWN(KEY_LEFT#) THEN position$ = position$ - speed$
IF KEYDOWN(KEY_RIGHT#) THEN position$ = position$ + speed$
```

The difference between `INKEY` and `KEYDOWN` is subtle but important. `INKEY` returns one key value per check - it is a snapshot. `KEYDOWN` can be called for multiple keys independently in the same frame, which means you can detect several keys being held simultaneously:

```
IF KEYDOWN(KEY_UP#)    THEN y$ = y$ - speed$
IF KEYDOWN(KEY_DOWN#)  THEN y$ = y$ + speed$
IF KEYDOWN(KEY_LEFT#)  THEN x$ = x$ - speed$
IF KEYDOWN(KEY_RIGHT#) THEN x$ = x$ + speed$
```

A player holding Up and Right at the same time will move diagonally. `INKEY` could not do that - it only returns one key.

Important: `KEYDOWN` only works in **graphics mode**. It requires a `SCREEN` command to have been called first. It will not function in a plain console program.

↑ Back to top

14.4 Key Constants

All three input methods work with the same set of built-in key constants. You have already seen `KEY_ESC#` and `KEY_1#`. Here is the full picture:

Constant	Key
`KEY_ESC#`	Escape
`KEY_ENTER#`	Enter
`KEY_SPACE#`	Space
`KEY_UP#` `KEY_DOWN#` `KEY_LEFT#` `KEY_RIGHT#`	Arrow keys
`KEY_F1#` … `KEY_F12#`	Function keys
`KEY_A#` … `KEY_Z#`	Letter keys
`KEY_0#` … `KEY_9#`	Number keys
`KEY_LSHIFT#` `KEY_LCTRL#`	Modifier keys

For printable characters, `CHR(key$)` converts the key value to its character - useful when you want to display what was pressed:

```
LET key$ = WAITKEY()
PRINT "You pressed: " + CHR(key$)
```

↑ Back to top

14.5 Mouse

The mouse gives you position and three buttons. All mouse input requires **graphics mode** - just like `KEYDOWN`, these only work after a `SCREEN` command.

```
LET x$ = MOUSEX       ' cursor X position in pixels
LET y$ = MOUSEY       ' cursor Y position in pixels
```

```
IF MOUSELEFT   THEN PRINT "Left button held"
IF MOUSERIGHT  THEN PRINT "Right button held"
IF MOUSEMIDDLE THEN PRINT "Middle button held"
```

The button values behave like `KEYDOWN` - they are 1 while the button is held, 0 when released. Check them each frame in your game loop:

```
[main]
    SCREEN 640, 480
    WHILE INKEY <> KEY_ESC#
        IF MOUSELEFT THEN
            CIRCLE (MOUSEX, MOUSEY), 5, 15, 1   ' draw dot where clicked
        END IF
        SLEEP 16
    WEND
END
```

Mouse input is straightforward. The coordinates track the cursor directly in pixel space, and the three button flags tell you what is being held at any moment.

↑ Back to top

14.6 Choosing the Right Tool

A quick summary:

Situation	Tool
Waiting for the user to type text	`INPUT` or `LINE INPUT`
Waiting for a specific keypress, nothing else to do	`WAITKEY`
Checking the keyboard each frame in a loop	`INKEY`
Detecting held keys or simultaneous keypresses	`KEYDOWN` (graphics only)
Reading cursor position and mouse buttons	`MOUSEX`, `MOUSEY`, `MOUSELEFT` etc. (graphics only)

The console tools - `INPUT`, `WAITKEY`, `INKEY` - work anywhere. The graphics tools - `KEYDOWN` and all mouse input - require a `SCREEN` to be open. Try to use them without one and they will not behave as expected.

↑ Back to top

15.0 Files

Everything your program does so far disappears the moment it ends. The score, the player's name, the settings - gone. Files are how a program remembers things between sessions.

BazzBasic keeps file handling simple. A handful of commands cover everything you will need for most programs.

↑ Back to top

15.1 Reading Files

`FileRead` reads the contents of a file into a variable:

```
LET data$ = FILEREAD("notes.txt")
PRINT data$
```

The entire file lands in `data$` as a single string. If the file does not exist, BazzBasic returns an empty string.

↑ Back to top

15.2 Writing Files

`FileWrite` creates a file - or overwrites it if it already exists:

```
FileWrite "notes.txt", "Hello from BazzBasic!"
```

If you want to add to an existing file without erasing what is already there, use `FileAppend` instead:

```
FileAppend "log.txt", "Program started.\n"
FileAppend "log.txt", "Player scored 100.\n"
```

Each call adds to the end. The file grows with each append.

↑ Back to top

15.3 Checking and Deleting

Before reading a file you are not certain exists, it is good practice to check first:

```
IF FileExists("save.txt") THEN
    LET data$ = FILEREAD("save.txt")
    PRINT "Save file found: " + data$
ELSE
    PRINT "No save file yet."
END IF
```

`FileExists` returns 1 if the file is there, 0 if it is not.

To remove a file entirely:

```
FileDelete "temp.dat"
```

↑ Back to top

15.4 Key-Value Files

Reading a whole file as one big string is useful, but BazzBasic can do something smarter. If you read a file into a `DIM`'d array instead of a plain variable, BazzBasic automatically parses lines in the `key=value` format into array entries:

```
DIM settings$
LET settings$ = FILEREAD("settings.txt")

PRINT settings$("playerName")
PRINT settings$("highScore")
```

The file `settings.txt` would look like this:

```
playerName=Alice
highScore=4200
```

Each line becomes an element in the array, accessible by its key. Lines starting with `#` are treated as comments and ignored - handy for adding notes to a settings file:

```
# Game settings
playerName=Alice
highScore=4200
```

Writing an array back out works the same way in reverse:

```
DIM settings$
settings$("playerName") = "Alice"
settings$("highScore")  = 4200
FileWrite "settings.txt", settings$
```

The file is created in `key=value` format automatically. Read it back with `FileRead` into a `DIM`'d array and you get exactly what you saved. The two round-trip perfectly.

↑ Back to top

15.5 The .env Pattern

The key-value format has one particularly useful application: configuration files. Many programs need settings that should not be hardcoded - API keys, usernames, server addresses. The convention is to store these in a file called `.env`:

```
# API configuration
API_KEY=abc123xyz
SERVER=https://api.example.com
```

Reading it in BazzBasic is simple:

```
DIM env$
LET env$ = FILEREAD(".env")
LET API_KEY# = env$("API_KEY")
LET SERVER#  = env$("SERVER")
```

Note that the values are stored as constants here - they are configuration that should not change while the program runs. This is the standard pattern and worth following.

One important thing: `.env` files often contain sensitive information like passwords or API keys. Never share them or include them in code you publish online. Keep them local, keep them private.

↑ Back to top

15.6 Paths

A few practical notes on how paths work in BazzBasic:

- Always use `/` or `\\` as the path separator - never a single `\`, as that is an escape character in strings
- Relative paths are relative to the folder where your program lives, which is also accessible as the built-in constant `PRG_ROOT#`

```
LET data$ = FILEREAD(PRG_ROOT# + "/data/save.txt")
```

- Absolute paths work too - just remember to escape the backslashes if you are on Windows:

```
LET data$ = FILEREAD("C:\\Users\\Alice\\Documents\\save.txt")
```

For most programs, relative paths are simpler and more portable. Keep your data files next to your program and you rarely need to think about paths at all.

↑ Back to top

15.7 A Practical Example

Here is a small program that saves and restores a player name and high score:

```
[inits]
    DIM save$
    LET playerName$ = "Anonymous"
    LET highScore$  = 0
    LET input$

[main]
    IF FileExists("save.txt") THEN
        LET save$ = FILEREAD("save.txt")
        playerName$ = save$("name")
        highScore$  = save$("score")
        PRINT "Welcome back, " + playerName$ + "!"
        PRINT "Your high score is " + highScore$ + "."
    ELSE
        LINE INPUT "First time here! Enter your name: ", playerName$
    END IF

    PRINT ""
    INPUT "Enter your score this run: ", input$

    IF input$ > highScore$ THEN
        highScore$ = input$
        PRINT "New high score!"
    END IF

    save$("name")  = playerName$
    save$("score") = highScore$
    FileWrite "save.txt", save$

    PRINT "Saved. See you next time, " + playerName$ + "."
END
```

Run it twice and the second time it remembers who you are. That is the whole idea.

↑ Back to top

16.0 JSON and Arrays

You already know BazzBasic arrays. You can store values by numeric index or by text key, mix both in the same array, and nest as many dimensions as you need. That flexibility is useful on its own - and it also makes JSON very natural in BazzBasic, because the two map directly onto each other.

↑ Back to top

16.1 What Is JSON

JSON is a text format for structured data. It is how most modern web services send and receive information, and how many programs store configuration or save files when key-value is not enough.

A simple JSON object looks like this:

```
{
  "name": "Alice",
  "score": 4200,
  "active": true
}
```

A JSON array looks like this:

```
["apple", "banana", "cherry"]
```

And they can nest - objects inside arrays, arrays inside objects, as deep as needed:

```
{
  "player": {
    "name": "Alice",
    "scores": [100, 200, 350]
  }
}
```

You do not need to write or parse JSON by hand in BazzBasic. Two commands handle the conversion in both directions.

↑ Back to top

16.2 Array to JSON

`ASJSON` converts a BazzBasic array into a JSON string:

```
DIM data$
data$("name")  = "Alice"
data$("score") = 4200

LET json$ = ASJSON(data$)
PRINT json$
```

Output:

```
{"name":"Alice","score":4200}
```

The array becomes a JSON object. Numeric keys become JSON arrays. String keys become object properties.

↑ Back to top

16.3 JSON to Array

`ASARRAY` goes the other direction - it parses a JSON string into a BazzBasic array:

```
DIM result$
LET json$ = "{""name"":""Alice"",""score"":4200}"
LET count$ = ASARRAY(result$, json$)

PRINT result$("name")   ' Alice
PRINT result$("score")  ' 4200
```

`ASARRAY` returns the number of elements it parsed, which you can use or ignore depending on your needs.

Note the doubled quotes `""` inside the string - that is BazzBasic's way of including a literal quote mark inside a string literal.

↑ Back to top

16.4 Nested Data

This is where BazzBasic's approach gets interesting. Nested JSON maps to arrays using **comma-separated keys**:

```
{
  "player": {
    "name": "Alice",
    "scores": [100, 200, 350]
  }
}
```

After parsing with `ASARRAY`, you access nested values like this:

```
DIM data$
LET count$ = ASARRAY(data$, json$)

PRINT data$("player,name")      ' Alice
PRINT data$("player,scores,0")  ' 100
PRINT data$("player,scores,1")  ' 200
PRINT data$("player,scores,2")  ' 350
```

Each level of nesting is separated by a comma in the key. Array positions are zero-based. Once you know the pattern, reading deeply nested JSON becomes straightforward.

Building nested JSON works the same way in reverse:

```
DIM body$
body$("model")              = "my-model"
body$("messages,0,role")    = "user"
body$("messages,0,content") = "Hello"

LET json$ = ASJSON(body$)
```

That `body$` array, when converted, produces properly nested JSON with a `messages` array containing one object. No manual string building required.

↑ Back to top

16.5 JSON Files

For saving and loading JSON files directly, BazzBasic has two dedicated commands that are cleaner than combining `FileWrite` with `ASJSON`:

```
' Save array as JSON file
DIM scores$
scores$("alice") = 4200
scores$("bob")   = 3100
SAVEJSON scores$, "scores.json"

' Load JSON file back into array
DIM loaded$
LOADJSON loaded$, "scores.json"
PRINT loaded$("alice")   ' 4200
```

`SAVEJSON` and `LOADJSON` are the file equivalent of `ASJSON` and `ASARRAY` - they just read and write directly to disk without the intermediate string step.

↑ Back to top

16.6 Putting It Together

Here is a small program that keeps a scoreboard in a JSON file - adding a new entry each run and showing the full list:

```
[inits]
    DIM board$
    LET playerName$
    LET playerScore$
    LET count$

[main]
    ' Load existing scoreboard if it exists
    IF FileExists("board.json") THEN
        LOADJSON board$, "board.json"
        count$ = board$("count")
    ELSE
        count$ = 0
    END IF

    ' Get this run's result
    LINE INPUT "Your name: ", playerName$
    INPUT "Your score: ", playerScore$

    ' Add new entry
    board$("entry," + count$ + ",name")  = playerName$
    board$("entry," + count$ + ",score") = playerScore$
    count$ = count$ + 1
    board$("count") = count$

    ' Save and display
    SAVEJSON board$, "board.json"

    PRINT ""
    PRINT "--- Scoreboard ---"
    FOR i$ = 0 TO count$ - 1
        PRINT board$("entry," + i$ + ",name") + ": " + board$("entry," + i$ + ",score")
    NEXT
END
```

Each time it runs, a new entry is added. The file grows. The board is always current.

↑ Back to top

17.0 Network

Your program can talk to the internet. Not in a complicated way - BazzBasic keeps it as simple as reading a file. Two commands cover almost everything: one for fetching data, one for sending it.

↑ Back to top

17.1 HTTPGET

`HTTPGET` sends a GET request to a URL and returns the response as a string:

```
LET response$ = HTTPGET("https://httpbin.org/get")
PRINT response$
```

That is all there is to it. The entire response body lands in `response$`. If the server returns JSON - which most modern APIs do - you can parse it straight into an array with `ASARRAY` from the previous chapter.

GET requests are for fetching data. You are asking the server for something, not sending it anything.

↑ Back to top

17.2 HTTPPOST

`HTTPPOST` sends a POST request with a body:

```
LET result$ = HTTPPOST("https://httpbin.org/post", "{""key"":""value""}")
PRINT result$
```

The second parameter is the request body - typically a JSON string. POST requests are for sending data. You are giving the server something to work with.

Building the body by hand with escaped quotes gets messy quickly. That is where `ASJSON` comes in:

```
DIM body$
body$("key") = "value"

LET result$ = HTTPPOST("https://httpbin.org/post", ASJSON(body$))
PRINT result$
```

Clean and readable.

↑ Back to top

17.3 Headers

Some services - most APIs - require additional headers alongside the request. Authentication tokens, content type declarations, API version identifiers. You pass these as an array:

```
DIM headers$
headers$("Authorization") = "Bearer mytoken"
headers$("Content-Type")  = "application/json"

LET result$ = HTTPGET("https://api.example.com/data", headers$)
```

The same optional headers parameter works with `HTTPPOST`:

```
LET result$ = HTTPPOST("https://api.example.com/data", ASJSON(body$), headers$)
```

↑ Back to top

17.4 Working With Responses

Responses from web APIs are almost always JSON. Parse them with `ASARRAY` and access fields the same way you learned in chapter 16:

```
DIM result$
LET count$ = ASARRAY(result$, HTTPGET("https://api.example.com/user"))

PRINT result$("name")
PRINT result$("email")
PRINT result$("address,city")
```

The nested comma-key notation handles any depth the response throws at you.

↑ Back to top

17.5 Putting It All Together

Here is where files, JSON, and network converge. This example calls the Anthropic Claude API - it reads an API key from a `.env` file, builds a JSON request body, sends it, and prints the response.

It is a short program. But it uses almost everything covered in the last three chapters.

This example expects you to have a *.env* file in a same folder as this code.

```
# .env
ANTHROPIC_API_KEY=sk-ant-yourKeyHere

# More about .env files: https://learn.microsoft.com/en-us/dynamics365/commerce/e-commerce-extensibility/configure-env-file
```

```
' ==========================================
' Anthropic Claude API call - BazzBasic
' https://github.com/EkBass/BazzBasic/blob/main/Examples/Anthropic_Claude_API_call.bas
' Public domain
' ==========================================
' Requires .env file in same directory:
'   ANTHROPIC_API_KEY=sk-ant-yourKeyHere
' ==========================================

[check:env]
    IF FileExists(".env") = 0 THEN
        PRINT "Error: .env file not found."
        END
    END IF

[inits]
    DIM env$
    env$ = FILEREAD(".env")
    LET ApiKey# = env$("ANTHROPIC_API_KEY")

    DIM headers$
    headers$("x-api-key")         = ApiKey#
    headers$("anthropic-version") = "2023-06-01"
    headers$("Content-Type")      = "application/json"

    DIM body$
    body$("model")              = "claude-model" ' adjust
    body$("max_tokens")         = 100
    body$("messages,0,role")    = "user"
    body$("messages,0,content") = "Hello"

[main]
    LET jsonBody$ = ASJSON(body$)
    LET raw$      = HTTPPOST("https://api.anthropic.com/v1/messages", jsonBody$, headers$)

    DIM result$
    LET count$ = ASARRAY(result$, raw$)

    PRINT result$("content,0,text")
END
```

Let's walk through it section by section.

`[check:env]` runs before anything else. If the `.env` file is missing, the program says so clearly and stops rather than crashing later with a confusing error. Checking preconditions early is a good habit.

`[inits]` loads the API key from `.env` as a constant - it will not change while the program runs. It then builds two arrays: `headers$` with the authentication and content-type fields the API requires, and `body$` with the actual request. The message is nested: `messages,0,role` and `messages,0,content` describe the first item in the `messages` array, exactly as the API expects.

`[main]` converts `body$` to a JSON string with `ASJSON`, sends it with `HTTPPOST` including the headers, then parses the response back into an array with `ASARRAY`. The reply text lives at `result$("content,0,text")` - the first item of the `content` array, its `text` field.

Twenty-five lines. A working API call to one of the most capable AI systems in the world.

↑ Back to top

18.0 Graphics and Sound

Everything so far has been text on a console. That is enough for a surprising range of programs - but BazzBasic can do considerably more. This chapter opens the graphics screen, draws things on it, and adds sound.

↑ Back to top

18.1 Opening a Screen

One command switches from console to graphics mode:

```
SCREEN 0, 640, 480, "My Window"
```

The parameters are mode, width, height, and window title. Mode 0 means custom size - the most common choice. If you prefer a preset, mode 12 gives you 640×480 without specifying dimensions:

```
SCREEN 12
```

From this point on, the console is gone and you are drawing pixels. `PRINT` still works - it writes text into the graphics window - but the real tools are the drawing commands.

↑ Back to top

18.2 Drawing

Clearing the screen

The fastest way to clear the screen between frames is a filled rectangle covering the whole window:

```
LINE (0, 0)-(SCREEN_W#, SCREEN_H#), 0, BF
```

`BF` means Box Filled. Color 0 is black. This is significantly faster than `CLS` in graphics mode - use it in every game loop.

Pixels

```
PSET (x$, y$), RGB(255, 255, 0)
```

A single pixel at position x, y in the given color. `RGB(r, g, b)` creates a color from red, green, and blue values each ranging from 0 to 255. For performance, store colors you use repeatedly in constants rather than calling `RGB` every frame.

Lines and boxes

```
LINE (x1$, y1$)-(x2$, y2$), color          ' line
LINE (x1$, y1$)-(x2$, y2$), color, B       ' box outline
LINE (x1$, y1$)-(x2$, y2$), color, BF      ' box filled
```

Circles

```
CIRCLE (cx$, cy$), radius$, color          ' outline
CIRCLE (cx$, cy$), radius$, color, 1       ' filled
```

↑ Back to top

18.3 Shapes and Images

For anything more complex than basic primitives, BazzBasic has shapes and images. Both work as handles - stable integer references stored in constants.

Creating a shape

```
LET BALL# = LOADSHAPE("CIRCLE", 30, 30, RGB(255, 255, 0))
```

Available shape types: `"CIRCLE"`, `"RECTANGLE"`, `"TRIANGLE"`. The dimensions follow, then the color.

Loading an image

```
LET PLAYER# = LOADIMAGE("player.png")
```

PNG with transparency and BMP are both supported.

Using shapes and images

```
MOVESHAPE BALL#, x$, y$     ' set position
ROTATESHAPE BALL#, angle$   ' rotate in degrees
SCALESHAPE BALL#, scale$    ' 1.0 = original size
DRAWSHAPE BALL#             ' render to screen
```

Always clean up when done:

```
REMOVESHAPE BALL#
```

↑ Back to top

18.4 SCREENLOCK

This is the most important habit in graphics programming. Always wrap your drawing code between `SCREENLOCK ON` and `SCREENLOCK OFF`:

```
SCREENLOCK ON
    LINE (0,0)-(SCREEN_W#, SCREEN_H#), 0, BF   ' clear
    MOVESHAPE BALL#, x$, y$
    DRAWSHAPE BALL#
SCREENLOCK OFF
```

`SCREENLOCK ON` starts buffering - nothing is shown on screen yet. `SCREENLOCK OFF` presents the finished frame all at once. Without it, the player sees each drawing operation as it happens, which produces visible flickering. With it, every frame appears clean and complete.

↑ Back to top

18.5 The Game Loop

Graphics programs follow a consistent pattern: check input, update state, draw, sleep, repeat.

```
[main]
    WHILE running$
        IF INKEY = KEY_ESC# THEN running$ = FALSE
        GOSUB [sub:update]
        GOSUB [sub:draw]
        SLEEP 16
    WEND
END

[sub:update]
    ' move things, handle logic
RETURN

[sub:draw]
    SCREENLOCK ON
        LINE (0,0)-(SCREEN_W#, SCREEN_H#), 0, BF
        ' draw everything
    SCREENLOCK OFF
RETURN
```

`SLEEP 16` gives roughly 60 frames per second. `INKEY` checks for keypresses without blocking - the loop keeps running whether or not anything is pressed. `KEYDOWN` is available for held keys and simultaneous input, but requires graphics mode - which you now have.

↑ Back to top

18.6 Sound

Sound in BazzBasic follows the same handle pattern as shapes. Load at startup, store as a constant, use the handle everywhere:

```
LET SND_JUMP# = LOADSOUND("jump.wav")
```

WAV and MP3 are both supported.

```
SOUNDONCE(SND_JUMP#)          ' play once, non-blocking
SOUNDONCEWAIT(SND_JUMP#)      ' play once, wait for finish
SOUNDREPEAT(SND_JUMP#)        ' loop continuously
SOUNDSTOP(SND_JUMP#)          ' stop this sound
SOUNDSTOPALL                  ' stop everything
```

Load all sounds at startup. Call `SOUNDSTOPALL` before `END` - always.

↑ Back to top

18.7 A Complete Example: Bouncing Ball

Everything in one program. A yellow ball bounces around the screen, reversing direction when it hits a wall.

```
' ==========================================
' Bouncing Ball - BazzBasic
' Press ESC to exit
' https://github.com/EkBass/BazzBasic/blob/main/Examples/BouncingBall.bas
' ==========================================

[inits]
    LET SCREEN_W#  = 640
    LET SCREEN_H#  = 480
    LET BALL_SIZE# = 30
    LET BALL_COL#  = RGB(255, 255, 0)

    LET x$       = 320
    LET y$       = 240
    LET dx$      = 3
    LET dy$      = 2
    LET running$ = TRUE

    SCREEN 0, SCREEN_W#, SCREEN_H#, "Bouncing Ball"
    LET BALL# = LOADSHAPE("CIRCLE", BALL_SIZE#, BALL_SIZE#, BALL_COL#)

[main]
    WHILE running$
        IF INKEY = KEY_ESC# THEN running$ = FALSE
        GOSUB [sub:update]
        GOSUB [sub:draw]
        SLEEP 16
    WEND
    REMOVESHAPE BALL#
END

[sub:update]
    x$ = x$ + dx$
    y$ = y$ + dy$

    IF x$ <= 0 OR x$ >= SCREEN_W# - BALL_SIZE# THEN dx$ = dx$ * -1
    IF y$ <= 0 OR y$ >= SCREEN_H# - BALL_SIZE# THEN dy$ = dy$ * -1
RETURN

[sub:draw]
    SCREENLOCK ON
        LINE (0,0)-(SCREEN_W#, SCREEN_H#), 0, BF
        MOVESHAPE BALL#, x$, y$
        DRAWSHAPE BALL#
    SCREENLOCK OFF
RETURN
```

The structure is exactly the pattern from 18.5. `[inits]` sets everything up including opening the screen and loading the shape. `[main]` is the game loop. `[sub:update]` moves the ball and flips its direction on collision. `[sub:draw]` clears the screen and renders the frame. `REMOVESHAPE` cleans up before exit.

↑ Back to top

18.8 FastTrig For Performance

Standard trig in BazzBasic works in radians:

```
LET x$ = SIN(RAD(45))
```

That is fine for occasional calculations. In a game loop calling trig hundreds of times per frame - a starfield, a raycaster, rotating sprites, particle systems - the conversion overhead adds up.

`FastTrig` replaces that with a lookup table. You enable it once, call `FastSin` and `FastCos` directly in degrees, and disable it when done:

```
FastTrig(TRUE)

LET x$ = FastCos(45)   ' no RAD() needed
LET y$ = FastSin(45)

FastTrig(FALSE)         ' free the memory
```

The accuracy is essentially identical to standard `SIN` / `COS`. The speed difference is significant - around 20 times faster for repeated calls. The table uses about 5.6 KB of memory while active, which is why you enable it only when you need it and disable it when you are done.

`FastRad` converts degrees to radians and does not require `FastTrig` to be enabled:

```
LET radians$ = FastRad(180)    ' = PI
```

Angles are automatically normalised - `FastSin(-90)` and `FastSin(450)` both work correctly, wrapping to the equivalent value in the 0–359 range.

A typical use in a game loop:

```
[inits]
    FastTrig(TRUE)
    LET angle$ = 0

[main]
    WHILE running$
        LET x$ = cx$ + radius$ * FastCos(angle$)
        LET y$ = cy$ + radius$ * FastSin(angle$)
        angle$ = MOD(angle$ + 1, 360)
        GOSUB [sub:draw]
        SLEEP 16
    WEND
    FastTrig(FALSE)
END
```

The rule is simple: if trig appears inside a loop that runs every frame, use `FastTrig`. If it is a one-off calculation, standard `SIN` and `COS` are fine.

↑ Back to top

18.9 What BazzBasic Can Do

The tools from this chapter - `SCREEN`, `LINE`, `PSET`, `FastTrig`, `SCREENLOCK`, `KEYDOWN`, arrays - are the same tools behind some impressive programs already in the examples folder. These are worth opening and exploring.

↑ Back to top

18.9.1 2D Field of View in console

`Fov_2D_demo.bas` https://github.com/EkBass/BazzBasic/blob/main/Examples/Fov_2D_demo.bas

A top-down maze with raycasting that runs entirely in the text console - no `SCREEN` command, no graphics mode. The player moves with arrow keys while 90 rays are cast each frame to calculate what cells are visible. Everything you see is `LOCATE`, `COLOR`, and `PRINT`. It is a good reminder that the console itself can do more than it seems.

↑ Back to top

18.9.2 Wolfenstein-style 3D Raycaster

`Raycaster_3d_optimized.bas` https://github.com/EkBass/BazzBasic/blob/main/Examples/Raycaster_3d_optimized.bas

A Wolfenstein-style first-person 3D renderer. The same map from the FOV demo, now rendered as vertical wall slices with distance-based shading, fish-eye correction, and a minimap overlay. `FastTrig` drives the per-frame ray math across 180 rays. The frame rate holds well even on modest hardware. The full renderer - movement, input, 3D projection, minimap - fits in a single `.bas` file.

↑ Back to top

18.9.3 Comanche-style Voxel Terrain

`Voxel_terrain.bas` https://github.com/EkBass/BazzBasic/blob/main/Examples/Voxel_terrain.bas

A Comanche-style heightmap renderer with procedurally generated terrain. Eight smoothing passes turn random noise into hills, valleys, water, beaches, forests, rock, and snow - each assigned a color by height band. The camera can move, rotate, change altitude, and tilt the horizon. It is the heaviest of the three in terms of computation, but still runs smoothly enough to feel like a real flyover.

↑ Back to top

19.0 Shelling Around

BazzBasic runs on Windows and hopefully in future at Linux too.

Both of them have a powerful command interpreter built in. `SHELL` connects the two — it sends a command to the system, waits for it to finish, and returns whatever it printed as a string.

```
' win
LET result$ = SHELL("dir *.txt")
PRINT result$
```

That is all there is to it. Anything you can type in a Windows command prompt, `SHELL` can run.

By default, `SHELL` waits up to 5 seconds for the command to finish. For commands that might take longer, pass a timeout in milliseconds as the second parameter:

```
LET result$ = SHELL("dir *.txt", 8000)
```

↑ Back to top

19.1 What You Can Do With It

List files

```
LET files$ = SHELL("dir *.bas /b")
PRINT files$
```

The `/b` flag gives bare output — filenames only, no headers or summaries. Clean and easy to work with.

Copy and move files

```
LET r$ = SHELL("copy save.txt backup.txt")
LET r$ = SHELL("move old.txt archive\old.txt")
```

Save output to a file

```
LET r$ = SHELL("dir *.bas /b > filelist.txt")
LET txt$ = FILEREAD("filelist.txt")
```

The `>` redirects output directly to a file. Combine it with `FileRead` and you have a file listing inside a BazzBasic variable.

Run another program

```
LET r$ = SHELL("notepad.exe")
```

↑ Back to top

19.2 Why This Matters

SHELL is the reason BazzBasic does not include built-in commands for everything. System information, zip compression, network pings — the command interpreter already does all of it. There is no point duplicating what is already there.

If you find yourself wanting a feature that BazzBasic does not have, ask yourself whether a *SHELL* command can do it. Quite often the answer is yes.

One thing to keep in mind: 'SHELL' executes commands using whatever terminal interpreter is available on the current system.

When BazzBasic supports Linux in the future, the same *SHELL* commands may not work on both systems.

ISWIN will be an important factor in this context. It tells you whether you are using Windows.

```
' ISWIN is planned to added at ver. 1.5 or 1.6'
IF ISWIN = TRUE THEN
    SHELL_EXECUTE# = "dir *.txt" ' win
ELSE
    SHELL_EXECUTE# = "ls *.txt" ' linux
END IF
LET temp$ = SHELL(SHELL_EXECUTE#)
```

↑ Back to top

19.2.1 Note About Linux Compatibility

At the time of writing this guide, BazzBasic has just advanced to version 1.2.

Technically, BazzBasic could be compiled for Linux already. C# + .NET10 + SDL2 are all fully compatible for Linux.

However, this is planned for version 1.5 or 1.6. I am mentioning it here now so the guide will not need updating when that version arrives.

↑ Back to top

20.0 BazzBasic Libraries

BazzBasic supports pre-compiled libraries *(.bb files)* for code reuse. Libraries contain tokenized functions that load faster than source files and use automatic name prefixes to prevent naming conflicts.

↑ Back to top

20.0.1 What is a Library?

A library is a collection of user-defined functions compiled into a binary format. When you create a library:

1. Source code is tokenized (converted to internal format)
2. Function names get a prefix based on the filename
3. The result is saved as a .bb file

↑ Back to top

20.0.2 Creating a Library

Step 1: Write your library source

Libraries can only contain **DEF FN** functions. No variables, constants, or executable code outside functions.

```
' MathLib.bas - A simple math library

DEF FN add$(x$, y$)
    RETURN x$ + y$
END DEF

DEF FN multiply$(x$, y$)
    RETURN x$ * y$
END DEF
```

Step 2: Compile the library This is not supported via BazzBasic built-in IDE. You must use terminal.

```
bazzbasic.exe -lib MathLib.bas
Output:

Created library: MathLib.bb
Library name: MATHLIB
Size: <size> bytes
```

The library name is derived from the filename (uppercase, without extension).

Step 3: Using a Library

Include the library

```
INCLUDE "MathLib.bb"
```

Step 4: Call library functions

Library functions are called with the FN keyword, using the library prefix:

```
INCLUDE "MathLib.bb"

LET a$ = 5
LET b$ = 3

PRINT "5 + 3 = "; FN MATHLIB_add$(a$, b$)
PRINT "5 * 3 = "; FN MATHLIB_multiply$(a$, b$)
```

```
Output:

5 + 3 = 8
5 * 3 = 15
```

↑ Back to top

20.0.3 Naming Convention

Source file	Library name	Function prefix
MathLib.bas	MATHLIB	MATHLIB_
StringUtils.bas	STRINGUTILS	STRINGUTILS_
MyGame.bas	MYGAME	MYGAME_

Original function *add$()* becomes *MATHLIB_add$()* when compiled from *MathLib.bas*.

↑ Back to top

20.0.4 Accessing Main Program Constants

Libraries cannot define constants, but library functions can access constants defined in the main program:

```
REM Main program
LET APP_NAME# = "MyApp"
LET APP_VERSION# = "1.0"

INCLUDE "Utils.bb"

PRINT FN UTILS_getAppInfo$()  ' Can use APP_NAME# and APP_VERSION#
```

```
REM Utils.bas - Library source

DEF FN getAppInfo$()
    RETURN APP_NAME# + " v" + APP_VERSION#
END DEF
```

This allows libraries to work with application-specific values without hardcoding them.

↑ Back to top

20.0.5 Library Rules

1. Functions only - Libraries cannot contain variables, constants, or loose code
2. Automatic prefix - All functions get the library name as prefix
3. No conflicts - Multiple libraries can have functions with the same base name
4. Binary format - .bb files are not human-readable

↑ Back to top

20.0.6 Error Handling

Invalid library content

```
bazzbasic.exe -lib BadLib.bas

Library validation failed:
  Line 5: Libraries can only contain DEF FN functions. Found: TOK_LET
```

Library not found

```
Error: Line 1: Library not found: missing.bb
```

↑ Back to top

20.0.7 Benefits of Libraries

1. Faster loading - Pre-tokenized code loads instantly
2. No name conflicts - Automatic prefixes prevent collisions
3. Code organization - Separate reusable code from main program
4. Smaller files - Binary format is typically 30-50% smaller than source when handling larger files

Note: For small files, i.e., files with only a few functions, the *.bas* file may be smaller than the tokenized *.bb* file. This is due to the metadata overhead in the tokenized format. But as the file size increases, the size advantage turns to the tokenized file.

↑ Back to top

20.0.8 Distributing Libraries

When distributing your program with libraries:

```
MyGame/
├── MyGame.exe      (or MyGame.bas)
├── MathLib.bb      (compiled library)
├── StringLib.bb    (compiled library)
└── SDL2.dll
```

Libraries must be in the same directory as the main program, or in a subfolder relative to it.

↑ Back to top

21.0 Compiling to EXE

BazzBasic can package your BASIC program into a standalone .exe file.

↑ Back to top

21.0.1 Usage

```
bazzbasic.exe -exe yourprogram.bas
```

This creates *yourprogram.exe* in the same folder as the .bas file.

↑ Back to top

21.0.2 Required Files for Distribution

The standalone *.exe* may need two additional files:

File	Required	Purpose
yourprogram.exe	Yes	Your packaged program
SDL2.dll	Yes	Graphics and input
SDL2_mixer.dll	Yes	Audio

SDL2.dll You need the *SDL2.dll* file if your program does not run entirely on the console. In other words, if your program uses graphics mode *(SCREEN)*, you need the *SDL2.dll* library included with your program.

SDL2_mixer.dll You need this file if your program uses any sound related features, such as *LoadSound()*

Your assets:

File	Required	Purpose
*.png, *.bmp	If used	Image files
*.wav, *.mp3	If used	Sound files

Minimal Distribution

```
mygame/
├── yourprogram.exe
├── SDL2_mixer.dll
└── SDL2.dll
```

↑ Back to top

21.0.3 Recommended Distribution with Assets

```
mygame/
├── yourprogram.exe
├── SDL2.dll
├── SDL2_mixer.dll
├── images/
│   ├── player.png
│   └── enemy.png
└── sounds/
    ├── shoot.wav
    └── music.mp3
```

↑ Back to top

21.0.4 Where to Find SDL2.dll

Binary download:

If you downloaded the binary *(BazzBasic.exe)*, the required SDL2 files are already in the same folder as bazzbasic.exe.

If you build from the source:

After building BazzBasic, SDL2.dll is in:

```
BazzBasic\bin\Debug\net10.0-windows\
```

Or after publishing:

```
BazzBasic\bin\Release\net10.0-windows\publish\
```

Copy SDL2.dll alongside your packaged exe.

↑ Back to top

21.0.5 Notes

- The packaged exe contains the full BazzBasic interpreter.
- File paths in your BASIC code are relative to the exe location.
- INCLUDE files are processed at packaging time (embedded in exe).

↑ Back to top

21.0.6 .NET Runtime Requirement

The packaged exe inherits the same runtime requirements as the BazzBasic.exe used to create it:

- **If using release build**: Target machine needs .NET 10 runtime
- **If using self-contained publish**: No runtime needed (larger file size)

To create a fully self-contained BazzBasic for packaging:

```
# This command is run in a terminal from the BazzBasic source folder.
dotnet publish -c Release -r win-x64 --self-contained true
```

Then use that published BazzBasic.exe for packaging your programs.

↑ Back to top

22.0 AI-Assisted Learning

I used the existing documentation and dozens of example programs that I gave to *Claude.AI*, *ChatGPT* and *Mistral Le Chat* for review.

The end result was the "BazzBasic-AI-guide.txt" file that they were all happy with.

This guide is not intended for casual human reading — its structure and content are specifically shaped for the needs of modern AI assistants.

By adding that guide to the context of the AI assistant you use — for example as a project file — it will have excellent information about BazzBasic and will thus be able to help, teach and generate code efficiently.

The guide is available for download on the Hugging Face server at: https://huggingface.co/datasets/EkBass/BazzBasic_AI_Guide/tree/main

I will update it as BazzBasic develops, so it is worth downloading the latest edition from time to time.

↑ Back to top

23.0 More Functions and Features

BazzBasic (ver. 1.2) includes nearly 100 functions and features and we simply cannot cover them all in this type of beginner's guide.

But for a beginner, there are a few that might be more interesting than others and I will list a few of them here.

↑ Back to top

23.1 Mathematical Functions

- BETWEEN(min$, max$) - Returns TRUE, if n is between min and max.
- CLAMP(n$, min$, max$) - If the given parameter n falls below or exceeds the given limit values min or max, it is returned within the limits.
- DISTANCE(x1$, y1$, x2$, y2$...) - Returns the Euclidean distance between two points. Supports both 2D and 3D.
- RND(n$) - Returns a random integer from 0 to n-1.
- LERP(start$, end$, t$) - Linear interpolation between two values. Returns a value between start and end based on parameter t (0.0 to 1.0).

See them all: Mathematical functions

↑ Back to top

23.2 String Functions

- BASE64DECODE(s$) & BASE64ENCODE(s$) - Decodes and encodes Base64
- INVERT(s$) - Inverts a string.
- MID(s$, start) or MID(s$, start, length) - Returns substring starting at position (1-based).
- REPEAT(s$, n) - Repeats the text requested times
- SPLIT(s$, a$, b$) - Splits string into array according to separator

See them all: String functions

↑ Back to top

23.3 Other Functions

- RGB(r$, g$, b$) - Creates a color value from red, green, and blue components (0-255 each).
- TIME(format$) - Returns current date/time as a formatted string. Uses .NET DateTime format strings.
- TICKS - Returns milliseconds elapsed since program started. Useful for timing, animations, and game loops.

↑ Back to top

23.4 Links

BazzBasic:

- BazzBasic homepage: https://ekbass.github.io/BazzBasic/
- BazzBasic manual: https://ekbass.github.io/BazzBasic/manual/#/
- BazzBasic source code: https://github.com/EkBass/BazzBasic

Communities and help:

- See a list of different communities where you can discuss and ask for help: https://ekbass.github.io/BazzBasic/communities.html
- GitHub discussions: https://github.com/EkBass/BazzBasic/discussions
- GotBasic Discord: https://discord.com/channels/682603735515529216/1464283741919907932
- Itch.io community: https://itch.io/board/5444321/basic-dialects

AI Guide:

- Formatted specifically as source material for AI assistants rather than casual reading: https://huggingface.co/datasets/EkBass/BazzBasic_AI_Guide

Example codes:

- GitHub: https://github.com/EkBass/BazzBasic/tree/main/Examples

Rosetta-Code:

- High-quality code for various tasks implemented by the Rosetta community: https://rosettacode.org/wiki/Category:BazzBasic

Other:

- BazzBasic readme.md file: https://github.com/EkBass/BazzBasic/blob/main/README.md
- BazzBasic changes.md file: https://github.com/EkBass/BazzBasic/blob/main/changes.md
- BazzBasic LICENSE.txt file: https://github.com/EkBass/BazzBasic/blob/main/LICENSE.txt

↑ Back to top

24.0 Final Words

Congratulations and thank you. You have now read through the entire guide and have certainly — or at least hopefully — learned what BazzBasic has to offer and what you can do with it.

But since the purpose of this guide is to get you started with BazzBasic rather than list the exact details of every function — you have already gotten there.

By reading the BazzBasic manual, you will get detailed information about dozens of mathematical and other functions and operations not mentioned in this guide.

Licensed under CC BY-ND 4.0.

- Free to share with attribution.
- Translations permitted.
- Modifications are not permitted.

```
PRINT "Made with BASIC-flavoured stubbornness and several cups of coffee."
PRINT "Kristian Virtanen (EkBass), author of BazzBasic"
END
```

↑ Back to top

EOF